

SIN 141 - Computação Orientada a Objetos - Lista 06

1. **Cadastro de Alunos** Uma escola precisa armazenar informações de seus alunos. Crie uma classe **Aluno** com os atributos **nome**, **matricula**, **curso** e **notaFinal**. Instancie três objetos dessa classe com dados fictícios e exiba as informações de cada aluno no console.
2. **Controle de Estoque** Uma pequena mercearia deseja controlar seus produtos. Modele uma classe **Produto** com os atributos **nome**, **preco**, **quantidade** e um método **calcularValorTotal()** que retorna o valor total em estoque daquele produto ($\text{preço} \times \text{quantidade}$). Crie dois objetos e exiba os resultados.
3. **Reserva de Hotel** Um sistema de reservas de hotel precisa que todo quarto seja criado já com suas informações básicas. Crie uma classe **Quarto** com os atributos **numero**, **tipo** (simples/duplo/suíte) e **precoPorNoite**. Implemente um construtor que exija todos esses dados no momento da criação do objeto. Instancie três quartos diferentes e exiba seus detalhes.
4. **Conta Bancária** Um banco exige que toda conta seja aberta com um titular e um saldo inicial. Crie uma classe **ContaBancaria** com os atributos **titular**, **numeroConta** e **saldo**. Implemente um construtor que receba esses valores e um método **exibirSaldo()**. Crie duas contas e mostre seus saldos iniciais.
5. **Calculadora de Área** Uma construtora precisa calcular a área de diferentes formas geométricas. Crie uma classe **Calculadora** com um método **calcularArea()** sobrecarregado para:
 - Receber um valor (área do quadrado)
 - Receber dois valores (área do retângulo)
 - Receber base e altura com um booleano indicando se é triângulo

Teste os três casos na classe principal.

6. **Funcionário com Múltiplos Cadastros** Uma empresa pode cadastrar funcionários com diferentes níveis de informação. Crie uma classe **Funcionario** com três construtores sobrecarregados:
 - Apenas **nome**
 - **nome** e **cargo**
 - **nome**, **cargo** e **salario**

Instancie um funcionário de cada forma e exiba seus dados, tratando valores não informados com um padrão (ex.: salário padrão de R\$ 1.500,00).

7. **Controle de Velocidade** Um sistema de monitoramento de veículos precisa garantir que a velocidade nunca seja negativa nem ultrapasse 200 km/h. Crie uma classe **Veiculo** com o atributo privado **velocidade** e implemente um **setVelocidade()** que valide essas regras, lançando uma mensagem de aviso caso o valor seja inválido. Teste com valores válidos e inválidos.
8. **Perfil de Usuário** Uma rede social precisa proteger os dados dos usuários. Crie uma classe **Usuario** com os atributos privados **nome**, **email** e **senha**. A senha deve ser armazenada com no mínimo 8 caracteres — o setter deve rejeitar senhas inválidas. Implemente getters para **nome** e **email**, mas **não** para **senha**. Teste o encapsulamento tentando definir uma senha curta.
9. **Sistema de Funcionários** Uma empresa possui três tipos de funcionários: **Gerente**, **Vendedor** e **Estagiario**, todos compartilhando atributos como **nome**, **cpf** e **salarioBase**. Crie uma classe pai **Funcionario** e as três subclasses, cada uma com um atributo exclusivo (ex.: Gerente tem **departamento**, Vendedor tem **metaMensal**, Estagiario tem **cargaHoraria**). Instancie um objeto de cada tipo e exiba seus dados.
10. **Hierarquia de Animais** Um zoológico deseja catalogar seus animais. Crie uma classe **Animal** com os atributos **nome**, **especie** e **idade**, e um método **emitirSom()**. Crie as subclasses **Cachorro**, **Gato** e **Passaro**, cada uma sobrescrevendo **emitirSom()** com o som correspondente. Instancie um objeto de cada subclasse e chame o método.
11. **Sistema de Pagamentos** Uma loja aceita diferentes formas de pagamento. Crie uma classe **Pagamento** com o método **processar(double valor)**. Crie as subclasses **PagamentoCartao**, **PagamentoPix** e **PagamentoBoleto**, cada uma sobrescrevendo **processar()** com uma mensagem específica (ex.: cartão aplica taxa de 2%). Crie uma lista de pagamentos, adicione um de cada tipo e processe todos em um laço.
12. **Relatório de Veículos** Uma locadora possui carros, motos e caminhões. Crie uma classe base **Veiculo** com o método **calcularCustoLocacao(int dias)**. Cada subclasse implementa o cálculo de forma diferente (ex.: carro R\$80/dia, moto R\$50/dia, caminhão R\$200/dia). Crie uma lista com os três veículos e exiba o custo de locação por 5 dias para cada um.
13. **Formas Geométricas** Um software de design precisa calcular a área e o perímetro de diferentes formas. Crie uma classe abstrata **Forma** com os métodos abstratos **calcularArea()** e **calcularPerimetro()**. Implemente as subclasses **Circulo**, **Retangulo** e **Triangulo**. Instancie uma de cada e exiba área e perímetro.
14. **Sistema de Notificações** Uma plataforma envia notificações por diferentes canais. Crie uma classe abstrata **Notificacao** com o método abstrato **enviar(String mensagem)** e um método concreto **registrarLog()**

que exibe a data e hora do envio. Implemente as subclasses

`NotificacaoEmail`, `NotificacaoSMS` e `NotificacaoPush`. Teste cada uma enviando uma mensagem.

15. **Dispositivos Inteligentes** Uma empresa de tecnologia fabrica dispositivos que podem se conectar via Wi-Fi e Bluetooth. Crie as interfaces `ConectavelWifi` (com método `conectarWifi()`) e `ConectavelBluetooth` (com método `conectarBluetooth()`). Crie a classe `SmartTV` que implementa ambas as interfaces. Instancie a `SmartTV` e demonstre as duas conexões.
16. **Funcionário Freelancer** Uma agência possui funcionários fixos e freelancers. Crie a interface `Pagavel` com o método `calcularPagamento()`. Crie a classe `FuncionarioFixo` (salário mensal fixo) e a classe `Freelancer` (valor por hora × horas trabalhadas), ambas implementando `Pagavel`. Crie uma lista do tipo `Pagavel`, adicione um de cada e exiba os pagamentos calculados.
17. **Robô Multifuncional** Uma fábrica possui robôs que realizam diferentes tarefas. Crie as interfaces `Soldador` (com método `soldar()`), `Pintor` (com método `pintar()`) e `Montador` (com método `montar()`). Crie a classe `RoboIndustrial` que implementa as três interfaces. Instancie o robô e execute todas as funções.
18. **Divisão Segura** Uma calculadora científica deve evitar erros em tempo de execução. Crie uma classe `Calculadora` com o método `dividir(double a, double b)` que lança uma exceção personalizada `DivisaoPorZeroException` quando o divisor for zero. Na classe principal, trate a exceção com `try-catch` exibindo uma mensagem amigável ao usuário.
19. **Saque Bancário** Um sistema bancário deve garantir que saques não ultrapassem o saldo disponível. Crie uma classe `ContaBancaria` com o método `sacar(double valor)` que lança uma exceção personalizada `SaldoInsuficienteException` quando o valor solicitado for maior que o saldo. Trate a exceção na classe principal e exiba o saldo atual após a tentativa.
20. **Cadastro com Validação Completa** Um sistema de cadastro de clientes deve validar todas as entradas. Crie uma classe `Cliente` com os atributos `nome`, `cpf` e `idade`. Implemente as seguintes validações com exceções personalizadas:
 - `NomeInvalidoException` — se o nome tiver menos de 3 caracteres
 - `CpfInvalidoException` — se o CPF não tiver 11 dígitos
 - `IdadeInvalidaException` — se a idade for menor que 0 ou maior que 120

Na classe principal, teste cada validação com `try-catch` e exiba mensagens claras para o usuário.